

Mémoire de fin d'études

Transformers et modèles de fondation pour les séries temporelles

Représentations, préentraînement et conditions du transfert

YOAN KESRAOUI

Année universitaire 2025 - 2026

Numéro étudiant 12517303

Septembre 2026

Résumé

Ce mémoire analyse les Transformers et les modèles de fondation pour la prévision des séries temporelles. Il relie les choix de représentation, les mécanismes d'attention, le préentraînement et la construction des sorties aux conditions du transfert vers de nouvelles séries. L'étude compare des architectures spécialisées et plusieurs familles préentraînées, puis examine la contamination des évaluations, les changements de distribution, l'adaptation et la dépendance entre horizons. La littérature conduit à considérer conjointement l'architecture, les informations accessibles et le protocole d'évaluation : une bonne précision moyenne ne suffit pas à établir une généralisation robuste. Quatre hypothèses expérimentales portent sur la prévision prospective, les covariables, l'adaptation limitée et les événements multi-horizons. Le travail constitue une revue critique et une proposition méthodologique ; les résultats cités proviennent des publications et aucune des expériences proposées n'a été réalisée dans ce mémoire.

Mots-clés : séries temporelles ; Transformers ; modèles de fondation ; préentraînement ; généralisation ; prévision probabiliste ; évaluation.

Abstract

This dissertation analyses Transformers and foundation models for time series forecasting. It connects input representations, attention mechanisms, pretraining and output construction to the conditions under which models transfer to new time series. The review compares specialised architectures and pretrained model families, then examines evaluation contamination, distribution shifts, adaptation and dependence across forecast horizons. The literature motivates a joint assessment of architecture, available information and evaluation design: strong average accuracy alone does not establish robust generalisation. Four experimental hypotheses address prospective forecasting, covariate quality, limited adaptation and multi-horizon events. This work is a critical literature review and a methodological proposal. All reported results originate from cited publications; none of the proposed experiments has been conducted as part of this dissertation.

Sommaire

1 Introduction ## 1.1 Sujet et problématique.....	5
1.2 Démarche et contribution.....	5
2 Comprendre le fonctionnement du Transformer	6
2.1 Représenter une séquence	6
2.2 Construire les requêtes, les clés et les valeurs	6
2.3 Calculer les poids et agréger les valeurs.....	7
2.4 Utiliser plusieurs têtes d'attention.....	8
2.5 Donner une information sur l'ordre	8
2.6 Empêcher l'accès au futur.....	9
2.7 Transformer et normaliser les représentations	9
2.8 Entraîner le réseau et produire une sortie	10
3 Adapter les Transformers aux séries temporelles	11
3.1 Définir la tâche avant de choisir une architecture.....	11
3.2 PatchTST et la représentation par segments	11
3.3 iTransformer et l'attention entre variables	12
3.4 Informer et la sélection des requêtes.....	13
3.5 Autoformer et les dépendances par décalage.....	13
3.6 Temporal Fusion Transformer et les covariables	13
3.7 Normaliser les fenêtres et conserver des comparateurs simples.....	14
3.8 Du modèle supervisé au modèle préentraîné.....	14
3.9 Mesurer la qualité des prévisions.....	15
4 Méthodologies des modèles de fondation temporels	17
4.1 Du préentraînement à une nouvelle prévision	17
4.2 Chronos et TimesFM face à la représentation des valeurs	17
4.3 Moirai et Chronos-2 pour structurer les variables	18
4.4 Sundial et la génération continue du futur	19
4.5 Les mélanges d'experts et la spécialisation des calculs.....	20
4.6 La construction des horizons longs	21
5 Limites des modèles et de leur évaluation	23
5.1 Vérifier ce que le modèle a déjà rencontré	23
5.2 Évaluer plusieurs tâches et des observations nouvelles.....	23
5.3 Comparer les informations disponibles et les coûts	24
5.4 Distinguer adaptation, oubli et robustesse	24
5.5 Tenir compte de la forme des prévisions	24

6 Pistesde recherche et expériences proposées	26
6.1 Cadre commun	26
6.2 H1 L'avantage zero-shot persiste après le gel du modèle.....	26
6.3 H2 Les covariables apportent un gain malgré une qualité imparfaite.....	26
6.4 H3 Une adaptation limitée réduit le coût sans perte excessive.....	26
6.5 H4 La dépendance améliore la prévision des événements multi-horizons.....	27
6.6 Portée du programme	27
7 Conclusion Cette revue montre que l'analyse des Transformers temporels doit relier les représentations aux informations accessibles et aux sorties attendues. Le choix des tokens organise les interactions ; le préentraînement détermine les régularités rencontrées ; le protocole précise ce que mesure réellement un score. Une bonne performance ne suffit donc pas à identifier le mécanisme du transfert.....	28
Bibliographie.....	29

1 Introduction ## 1.1 Sujet et problématique

Prévoir une série temporelle demande d'extraire des régularités d'un historique sans supposer qu'elles resteront inchangées. Une saisonnalité peut évoluer, une variable devenir indisponible et une relation entre mesures disparaître. Les modèles de fondation cherchent à réutiliser des connaissances acquises sur de nombreuses séries, parfois sans mise à jour de leurs paramètres sur la tâche cible. Cette ambition de transfert motive leur étude, mais ne constitue pas une garantie de généralisation.

Le Transformer fournit un mécanisme flexible de mise en relation des observations. Son comportement dépend toutefois de ce que représentent les tokens, des interactions autorisées et de l'objectif d'apprentissage. Des valeurs discrétisées, des segments continus et des variables entières ne décrivent pas le même problème au réseau. De même, prédire des quantiles par horizon et produire des trajectoires conjointes répond à des besoins différents.

La problématique est donc la suivante : **quels choix de représentation et de préentraînement permettent un transfert utile vers de nouvelles séries, et quels protocoles permettent de l'établir ?** Elle articule l'analyse des mécanismes à celle des preuves empiriques. Un gain peut provenir d'une meilleure architecture, mais aussi d'informations supplémentaires, d'une proximité avec le corpus appris ou d'un budget plus élevé.

L'évaluation doit tenir compte de la dépendance temporelle et des informations disponibles à chaque origine de prévision. Elle doit aussi distinguer la réutilisation de connaissances sur des archives du transfert vers des observations réellement futures. Ces distinctions guideront l'analyse des résultats publiés et la formulation des expériences proposées.

1.2 Démarche et contribution

La revue sélectionne des méthodes illustrant des choix distincts de représentation, d'architecture et d'apprentissage. Pour chaque famille, elle suit les entrées, les calculs, l'objectif et les sorties. Les chapitres 2 et 3 exposent les mécanismes et leurs adaptations temporelles ; le chapitre 4 étudie le préentraînement ; le chapitre 5 analyse les limites des évaluations. Le chapitre 6 transforme ces limites en hypothèses réfutables.

La contribution réside dans cette grille de comparaison et dans les contrôles expérimentaux proposés. Les articles d'origine sont privilégiés ; les rapports techniques sont distingués des publications évaluées. Le corpus couvre les travaux consultés jusqu'au 9 septembre 2026. La sélection est centrée sur la prévision temporelle et ne constitue pas une revue systématique exhaustive. Ce travail théorique et bibliographique ne présente aucune nouvelle architecture validée ni aucun résultat expérimental original.

2 Comprendre le fonctionnement du Transformer

2.1 Représenter une séquence

Le Transformer transforme une suite d'éléments en une suite de vecteurs contextualisés. Pour une position donnée, la représentation obtenue dépend de son contenu et des autres éléments accessibles. Cette mise en relation est assurée par l'attention. Le réseau comporte également des transformations non linéaires, des connexions résiduelles et des normalisations. Ces opérations sont répétées dans plusieurs couches (Vaswani et al., 2017).

On note n le nombre d'éléments de la séquence, d_{model} la dimension de leur représentation et h le nombre de têtes d'attention. Il faut d'abord choisir ce que représente un élément, appelé token. Dans un texte, ce peut être un mot ou une partie de mot. Dans une série temporelle, ce peut être une mesure, un vecteur de variables ou un segment de plusieurs mesures. Ce choix détermine les informations que le réseau pourra directement mettre en relation.

Pour un token discret d'indice v_i , l'embedding consiste à sélectionner une ligne dans une matrice E apprise. L'indice sert uniquement à retrouver un vecteur ; sa valeur numérique ne mesure pas une proximité entre tokens. Pour une entrée numérique u_i , une projection affine permet de construire directement le vecteur. On peut ensuite ajouter un vecteur de position p_i .

$$x_i = E[v_i] + p_i \text{ ou } x_i = u_i W_E + b_E + p_i \quad (1)$$

Le vecteur numérique u_i est projeté par W_E ; p_i représente sa position. Les biais et les positions ont d_{model} coordonnées.

Les représentations sont regroupées dans une matrice X de taille $n \times d_{model}$. Chaque ligne correspond à un token. Dans la suite, les formules utilisent cette convention de vecteurs lignes. Les dimensions sont importantes : elles permettent de vérifier quelles opérations sont possibles et sur quel axe l'information circule.

2.2 Construire les requêtes, les clés et les valeurs

L'auto-attention commence par trois projections linéaires de X . Elles produisent les requêtes Q , les clés K et les valeurs V . Les matrices de projection sont des paramètres appris avec le reste du réseau.

$$Q = XW_Q, K = XW_K, V = XW_V \quad (2)$$

Q et K ont $n \times d_k$ coordonnées ; V en a $n \times d_v$. Les biais sont omis.

La requête q_i décrit ce qui sert à chercher de l'information pour la position i . Elle est comparée à chaque clé k_j . La valeur v_j contient, quant à elle, le vecteur transmis lorsque l'élément j reçoit un poids important. Cette explication aide à distinguer les rôles, mais il ne faut pas imaginer que le réseau formule des questions en langage naturel.

Dans l'auto-attention, Q , K et V proviennent de la même séquence. Elles restent différentes puisque leurs matrices de projection sont distinctes. Pour une attention croisée, on utilise deux séquences : les requêtes sont construites à partir de la première, les clés et les valeurs à partir de la seconde. Ce mécanisme permet, par exemple, à un décodeur de consulter une entrée déjà encodée.

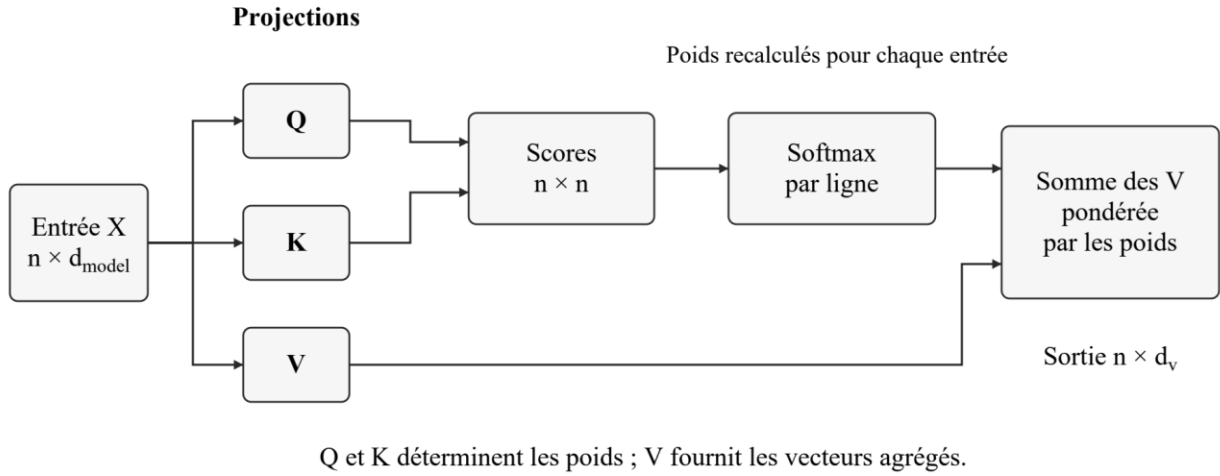


Figure 1 Calcul d'une tête d'auto-attention. Schéma original d'après Vaswani et al. (2017).

2.3 Calculer les poids et agréger les valeurs

Le produit QK^T mesure toutes les compatibilités entre les requêtes et les clés. Dans le cas d'une auto-attention sur n éléments, il produit une matrice $n \times n$. Le score de la ligne i et de la colonne j est ensuite divisé par la racine carrée de d_k , la dimension des requêtes et des clés. La fonction softmax est appliquée sur les colonnes de chaque ligne. Elle transforme les scores en coefficients positifs dont la somme vaut un.

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (3)$$

Le softmax s'applique aux colonnes de chaque ligne. La sortie a la taille $n \times d_v$.

Le calcul peut être détaillé pour une position. On obtient d'abord le score s_{ij} , puis le coefficient a_{ij} , et enfin la somme pondérée des valeurs. La formule suivante suppose qu'aucune position n'est masquée.

$$s_{ij} = \frac{q_i k_j^T}{\sqrt{d_k}}, a_{ij} = \frac{\exp(s_{ij})}{\sum_{\ell=1}^n \exp(s_{i\ell})}, y_i = \sum_{j=1}^n a_{ij} v_j \quad (4)$$

Forme sans masque ; la somme des a_{ij} vaut un pour chaque requête i .

Prenons un exemple simple : si les poids d'une requête sont 0,7, 0,2 et 0,1, sa sortie vaut $0,7v_1 + 0,2v_2 + 0,1v_3$. Il s'agit d'une moyenne pondérée de vecteurs, pas nécessairement d'une moyenne des observations brutes. Ces poids changent avec l'entrée, alors que les matrices de projection restent fixes pendant une inférence ordinaire. Cette distinction explique comment le même réseau peut exploiter différemment deux historiques.

La division par $\sqrt{d_k}$ évite que les scores deviennent trop grands quand leur dimension augmente. Si l'on suppose, pour comprendre ce point, des composantes indépendantes, centrées et de variance un, la variance du produit scalaire augmente avec d_k . Des scores de grande amplitude peuvent concentrer fortement le softmax et réduire ses gradients. L'argument motive le changement d'échelle ; il n'impose pas que les composantes apprises restent indépendantes.

Les coefficients d'attention décrivent une opération du réseau. Ils ne suffisent pas à établir une relation causale entre variables ni à expliquer toute la prédiction. La sortie dépend aussi des valeurs projetées et des couches qui suivent.

2.4 Utiliser plusieurs têtes d'attention

Une attention à plusieurs têtes répète ce calcul avec des projections différentes. Chaque tête produit une représentation ; les résultats sont concaténés puis projetés dans la dimension d_{model} . Le réseau peut ainsi calculer plusieurs types de relations à partir de la même entrée.

$$MHA(X) = Concat(Y_1, \dots, Y_h)W_O \quad (5a)$$

$$Y_r = Attention(XW_Q^{(r)}, XW_K^{(r)}, XW_V^{(r)}) \quad (5b)$$

La projection W_O transforme $h \times d_v$ coordonnées en d_{model} coordonnées.

Le choix fréquent $d_k = d_v = d_{model}/h$ conserve une dimension totale constante pour l'ensemble des têtes. Avec $d_{model} = 128$ et $h = 8$, chaque tête peut par exemple produire 16 coordonnées. Leur concaténation en contient alors 128. Ajouter des têtes ne signifie donc pas nécessairement multiplier la dimension finale.

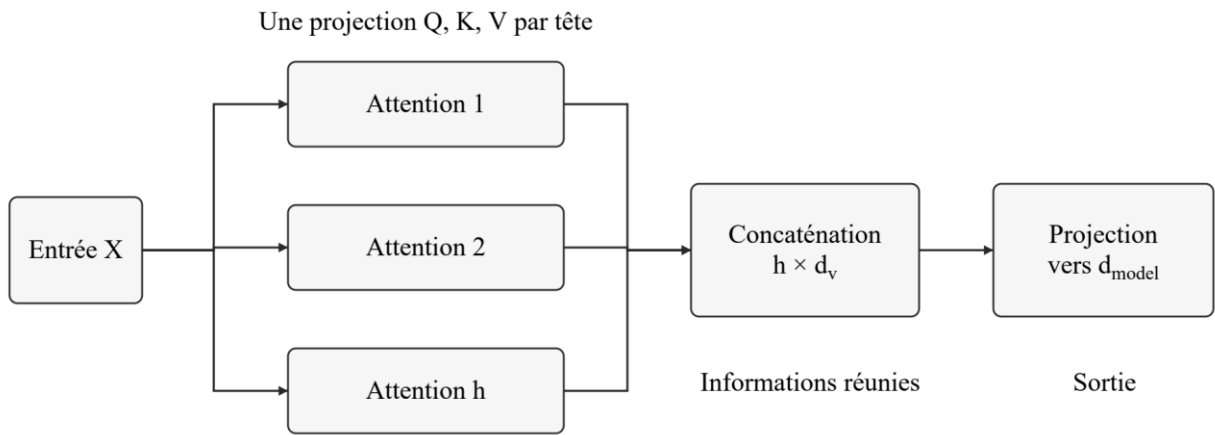


Figure 2 Attention à plusieurs têtes. Schéma original d'après Vaswani et al. (2017). Chaque tête utilise ses propres projections.

Les paramètres de chaque tête sont appris. On peut étudier leur spécialisation après l'entraînement, mais rien ne garantit qu'une tête représentera exclusivement une saisonnalité et une autre une tendance. Pour montrer qu'une tête joue un rôle particulier, il faut examiner son comportement ou mesurer l'effet de son retrait.

2.5 Donner une information sur l'ordre

L'attention calculée uniquement à partir du contenu ne distingue pas spontanément les positions. Sans encodage positionnel ni masque, une permutation des lignes de X entraîne la même permutation des lignes de sortie : le calcul est équivariant aux permutations. Pour utiliser l'ordre, le modèle doit donc recevoir une information supplémentaire.

Le Transformer original utilise notamment un encodage sinusoïdal ajouté aux embeddings. Les coordonnées paires et impaires représentent des sinus et des cosinus de fréquences différentes.

$$PE(p, 2r) = \sin\left(\frac{p}{10000^{2r/d_{model}}}\right) \quad (6a)$$

$$PE(p, 2r + 1) = \cos\left(\frac{p}{10000^{2r/d_{model}}}\right) \quad (6b)$$

Pour d_{model} pair, $r = 0, \dots, d_{model}/2 - 1$; p est une position entière à partir de zéro.

Cette construction fournit un vecteur pour chaque position p . Certaines coordonnées varient rapidement, d'autres plus lentement. La formule reste calculable pour des positions plus grandes que celles rencontrées pendant l'apprentissage, sans garantir que le modèle saura exploiter un contexte beaucoup plus long.

Pour une série temporelle, la position dans une fenêtre doit aussi être distinguée de la date. Le dixième point peut correspondre à dix heures ou à dix jours selon l'échantillonnage. Si les mesures sont irrégulières, deux positions consécutives ne représentent même pas toujours la même durée. Des variables calendaires ou des écarts de temps peuvent donc compléter l'encodage des positions.

2.6 Empêcher l'accès au futur

Pendant l'apprentissage d'un décodeur autorégressif, les entrées sont décalées par rapport aux cibles. La position i utilise un élément déjà connu pour prédire le suivant. Un masque causal lui permet de consulter les positions $j \leq i$ et interdit les suivantes. Il est ajouté aux scores avant le softmax.

$$M_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases} \quad (7a)$$

$$Y = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V \quad (7b)$$

M a la taille $n \times n$. Cette convention suppose des entrées du décodeur décalées.

Le masque a la taille $n \times n$ et porte sur les relations entre positions. Avec la convention décalée, la diagonale est autorisée. La valeur $-\infty$ annule le poids d'une position interdite après le softmax. Il faut conserver au moins une clé accessible par requête pour que ce calcul reste défini. Un masque de remplissage peut être ajouté pour ignorer les emplacements artificiels utilisés dans un lot de séquences de longueurs différentes.

En prévision, un encodeur qui reçoit uniquement le passé peut consulter toute cette fenêtre sans fuite. La règle essentielle est de ne pas lui fournir une information indisponible à l'origine de la prévision. Un masque triangulaire dans toutes les couches n'est donc pas une condition nécessaire pour tous les modèles temporels.

2.7 Transformer et normaliser les représentations

L'attention fait circuler l'information entre positions. Le réseau feed-forward, ou FFN, transforme ensuite chaque vecteur séparément avec les mêmes paramètres à toutes les positions d'une couche. Une forme classique utilise deux projections affines et une activation ReLU.

$$FFN(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2 \quad (8)$$

W_1 projette de d_{model} vers d_{ff} ; W_2 revient de d_{ff} vers d_{model} .

Une connexion résiduelle ajoute l'entrée d'un sous-bloc à sa sortie. Le sous-bloc apprend ainsi une modification de la représentation existante. Cette voie directe aide à transmettre l'information et les gradients lorsque plusieurs couches sont empilées. Elle impose aussi que les deux termes additionnés aient la même taille.

La normalisation par couche, appelée LayerNorm, utilise la moyenne et la variance des coordonnées d'un token. Elle recentre le vecteur, le remet à l'échelle et applique des coefficients

appris γ et β . Le terme ε évite une division instable lorsque la variance est très faible (Ba et al., 2016).

$$LN(z) = \gamma \odot \frac{z - \mu(z)}{\sqrt{\sigma^2(z) + \varepsilon}} + \beta \quad (9a)$$

$$\mu(z) = \frac{1}{d_{\text{model}}} \sum_{j=1}^{d_{\text{model}}} z_j, \sigma^2(z) = \frac{1}{d_{\text{model}}} \sum_{j=1}^{d_{\text{model}}} (z_j - \mu(z))^2 \quad (9b)$$

γ et β sont appris ; $\varepsilon > 0$. Les statistiques portent sur les coordonnées d'un token.

LayerNorm ne normalise pas la série brute sur l'axe du temps. Elle agit sur les coordonnées d'une représentation, à une position donnée. Dans le bloc original, elle suit l'addition résiduelle. D'autres architectures la placent avant le sous-bloc. Les deux organisations ne doivent pas être confondues.

$$Z = LN(X + MHA(X)), X' = LN(Z + FFN(Z)) \quad (10)$$

Bloc post-normalisation ; dropout omis. FFN et LN s'appliquent ligne par ligne.

2.8 Entraîner le réseau et produire une sortie

L'encodeur empile des blocs pour représenter l'entrée. Dans l'architecture encodeur-décodeur, le décodeur possède une auto-attention masquée et une attention croisée vers les sorties de l'encodeur. Un modèle à encodeur seul peut recevoir une tête de prévision ; un modèle à décodeur seul peut prolonger un contexte de façon causale. Le Transformer décrit donc une organisation du calcul, à laquelle il faut ajouter un objectif d'apprentissage et une sortie adaptée.

Pendant l'entraînement, une fonction de perte compare la sortie aux cibles. La rétropropagation calcule les gradients par rapport aux embeddings, aux projections d'attention, aux FFN et à la tête de sortie. Un optimiseur met à jour ces paramètres. Les coefficients d'attention sont recalculés à chaque passage : ils ne forment pas une matrice de paramètres conservée pour toutes les séquences.

Pendant l'apprentissage autorégressif, le décodeur reçoit les tokens réels décalés. À l'inférence, il utilise progressivement ses propres sorties, ce qui peut propager les erreurs. Une tête qui produit directement tous les horizons suit une autre procédure.

Le coût de l'attention dense croît quadratiquement avec n à dimensions fixées. La version standard matérialise h matrices de scores $n \times n$; les projections et les FFN ajoutent leur propre coût. Enfin, pouvoir entraîner plusieurs positions en parallèle ne supprime pas les étapes successives d'une génération autorégressive. Les adaptations temporelles cherchent justement à réduire le nombre de tokens, à modifier les relations calculées ou à produire plusieurs horizons dans un même passage.

3 Adapter les Transformers aux séries temporelles

3.1 Définir la tâche avant de choisir une architecture

La prévision consiste à exploiter un historique pour estimer des observations qui ne sont pas encore disponibles. On note L la longueur de cet historique, H l'horizon et C le nombre de variables cibles. À une date t , les données d'entrée forment donc une matrice de L lignes et C colonnes. Ces dimensions ne sont pas interchangeables : augmenter L étend la mémoire temporelle, tandis qu'augmenter C ajoute des variables susceptibles d'apporter une information complémentaire.

$$\hat{y}_{t+1:t+H} = f_{\theta}(y_{t-L+1:t} \ x_{t-L+1:t} \ z_{t+1:t+H} \ s) \quad (11)$$

Dans cette écriture, y représente les cibles, x les covariables historiques, z les covariables futures connues à la date t et s les caractéristiques statiques. Les paramètres θ sont appris. Un calendrier futur peut être fourni au modèle ; une mesure réalisée après t ne peut pas l'être. De même, lorsqu'une covariable provient d'une autre prévision, il faut utiliser la version disponible à l'origine de prévision. Sans cette précaution, une architecture peut paraître performante simplement parce qu'elle reçoit une information inaccessible en pratique.

L'apprentissage se construit en faisant glisser des fenêtres de longueur $L + H$ sur les séries. Les L premières observations servent d'entrée et les H suivantes fournissent la cible. On calcule une fonction de perte, puis la rétropropagation ajuste les paramètres. La séparation entre apprentissage, validation et test doit respecter la chronologie ; les fenêtres réservées au test ne servent ni à choisir les hyperparamètres ni à estimer les transformations globales des données.

Pour comparer les méthodes, on retient le même fil conducteur : comment les données sont préparées, quels éléments deviennent des tokens, quelles informations échantillonnent ces tokens, puis comment le réseau produit la prévision. Cette lecture permet de comprendre ce que modifie réellement chaque architecture.

3.2 PatchTST et la représentation par segments

PatchTST normalise chaque canal, puis découpe son historique en segments de longueur P , espacés d'un pas S . Une projection transforme les segments en vecteurs de dimension d ; un encodage conserve leurs positions. La version supervisée ajoute S répétitions de la dernière valeur (Nie et al., 2023).

$$N = \left\lfloor \frac{L-P}{S} \right\rfloor + 2 \quad (12a)$$

$$E_c = P_c W_p + E_{pos} \quad (12b)$$

P_c a la taille $N \times P$; W_p a la taille $P \times d$; E_c et E_{pos} ont la taille $N \times d$. La formule inclut le remplissage final de S valeurs.

N compte les segments, P_c est leur matrice pour le canal c , W_p la projection et E_{pos} l'encodage de position. L'attention relie les segments d'un même canal. Les paramètres sont partagés entre canaux, sans échange direct entre leurs représentations. Après l'attention, les réseaux feed-forward et les normalisations BatchNorm, la représentation aplatie est projetée vers H valeurs. L'entraînement minimise une erreur quadratique ; l'inférence produit l'horizon directement.

Dans un exemple pédagogique avec $L = 96$, $P = 16$ et $S = 8$, on obtient 12 tokens par canal. Une tête construit alors une matrice d'attention de 12×12 . Le choix de P détermine les détails locaux regroupés avant l'attention ; celui de S règle le recouvrement et le nombre de tokens.

3.3 iTransformer et l'attention entre variables

iTransformer change l'axe sur lequel l'attention opère. L'historique complet d'une variable est projeté vers un token de dimension d . Les C tokens obtenus traversent un encodeur dont l'attention combine les représentations des variables. Le réseau feed-forward, partagé entre tokens, transforme ensuite chaque représentation. Une projection finale produit les H valeurs futures de chaque variable, et l'apprentissage supervisé utilise l'erreur quadratique (Liu et al., 2024).

$$e_c^{(0)} = E(U_{:,c}) \quad (13a)$$

$$Z^{(\ell+1)} = B_\ell(Z^{(\ell)}), \ell = 0, \dots, K-1 \quad (13b)$$

$$\hat{Y}_{:,c} = G(Z_{c,:}^{(K)}) \quad (13c)$$

$Z^{(0)}$ réunit les C tokens initiaux. Les matrices Z ont la taille $C \times d$ et $\hat{Y}$ la taille $H \times C$. La restitution de l'échelle est omise dans ce schéma latent.

E transforme un historique de longueur L en un vecteur ; B_ℓ désigne le bloc numéro ℓ et G la projection vers l'horizon. Le nombre de blocs est K . Dans un exemple pédagogique avec $C = 8$ et $d = 64$, l'encodeur reçoit une matrice de 8×64 , même si chaque variable possède 96 observations. L'attention compare alors huit historiques représentés par huit tokens, à travers une matrice de 8×8 .

Cette attention coûte quadratiquement en C ; l'encodage dépend toujours de L . Les positions dans l'historique projeté conservent l'ordre temporel. Les relations entre variables sont explicites, sans comparaison de toutes les paires d'instants.

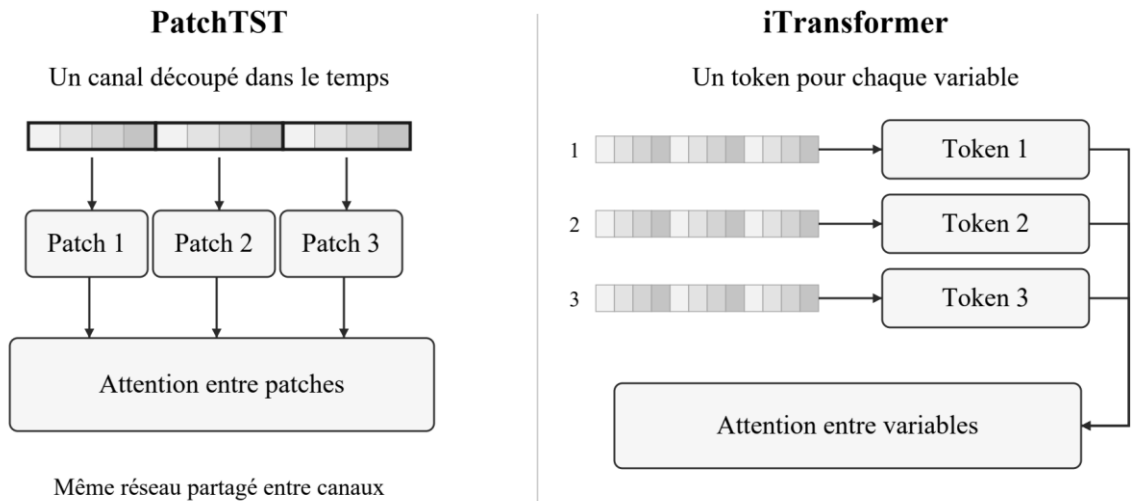


Figure 3 Deux façons de construire les tokens. Schéma original d'après Nie et al. (2023) et Liu et al. (2024). Le découpage est illustratif ; le remplissage en bord de fenêtre est omis.

3.4 Informer et la sélection des requêtes

Informer conserve une organisation encodeur-décodeur, mais réduit le calcul d'attention avec ProbSparse. Son principe est de consacrer le calcul détaillé aux requêtes dont les scores se distinguent le plus d'une distribution uniforme. Une approximation fondée sur un échantillonnage de clés estime cette propriété ; seules les requêtes retenues calculent ensuite une attention complète sur les clés. Dans l'encodeur, les autres positions conservent un contexte initial fondé sur les valeurs (Zhou et al., 2021).

Entre certains blocs, une convolution temporelle, une activation et un max-pooling réduisent la longueur de la représentation. Cette opération, appelée distillation, diminue le nombre de positions à traiter aux étages suivants. Le décodeur reçoit une portion récente de l'historique et des emplacements nuls pour les valeurs futures, accompagnés des informations temporelles disponibles. Une passe produit tout l'horizon ; l'apprentissage minimise l'erreur quadratique.

La complexité annoncée en $O(L \log L)$ concerne le mécanisme proposé. La méthode repose donc sur deux réductions différentes : sélectionner les requêtes et raccourcir les représentations. Pour comprendre une variation de performance, il faudrait les retirer séparément. Une sélection trop forte peut supprimer une dépendance utile, tandis que le pooling peut atténuer un événement bref.

3.5 Autoformer et les dépendances par décalage

Autoformer introduit la décomposition à l'intérieur du réseau. Un lissage par moyenne mobile extrait une composante lente ; la différence avec l'entrée constitue un résidu appelé composante saisonnière. Cette appellation décrit le rôle du bloc : elle ne garantit pas que le résidu soit une saisonnalité pure. Les décompositions sont répétées sur les représentations intermédiaires, et le décodeur accumule progressivement les contributions de tendance (Wu et al., 2021).

Le mécanisme Auto-Correlation remplace l'attention point par point. Les projections Q et K servent à mesurer des similarités pour plusieurs décalages temporels. Le calcul utilise des transformées de Fourier rapides. Le modèle retient les décalages les plus pertinents, décale circulairement les représentations V, puis additionne ces versions avec des poids normalisés par softmax. Il rapproche ainsi des sous-séries alignées selon un retard commun.

La sortie additionne les composantes prédites sur l'horizon, avec un apprentissage par erreur quadratique. L'hypothèse structurante est que des motifs décalés apportent une information réutilisable. Elle donne un sens temporel aux interactions apprises, mais invite à tester la sensibilité aux changements de période. La largeur du lissage et le nombre de retards retenus constituent également des choix méthodologiques à documenter.

3.6 Temporal Fusion Transformer et les covariables

Le Temporal Fusion Transformer, ou TFT, organise explicitement les variables statiques, historiques et futures connues. Les variables catégorielles passent par des embeddings et les variables continues par des projections. Des réseaux de sélection calculent des poids normalisés par softmax, puis combinent les représentations des variables. Ce choix dépend des entrées et, pour les variables temporelles, du contexte statique (Lim et al., 2021).

Des couches LSTM traitent ensuite les dépendances locales sur l'historique et les entrées futures disponibles. Les représentations sont enrichies par les informations statiques, puis une attention temporelle masquée combine les positions admissibles. Des portes apprises règlent la

contribution des transformations non linéaires dans les connexions résiduelles. L'attention modifiée partage la projection des valeurs entre les têtes et agrège leurs contributions, afin de faciliter leur lecture.

La tête de sortie fournit plusieurs quantiles pour chaque horizon. La fonction de perte est donc la perte quantile, présentée dans la section sur les métriques, plutôt qu'une simple erreur quadratique. Le modèle produit ces quantiles directement à partir des entrées disponibles. Les poids de sélection renseignent sur son fonctionnement, mais leur amplitude ne suffit pas à établir une relation causale entre une covariable et la cible.

3.7 Normaliser les fenêtres et conserver des comparateurs simples

RevIN peut entourer un modèle de prévision. Pour chaque fenêtre et chaque canal, on calcule une moyenne μ_c et une échelle s_c , définie à partir de la variance et d'une petite constante ε . La fenêtre est centrée et réduite ; après la prévision, les mêmes statistiques restaurent les unités initiales. Dans la forme sans transformation affine apprise, on obtient les relations suivantes (Kim et al., 2022).

$$\tilde{y}_{r,c} = \frac{y_{r,c} - \mu_c}{s_c} \quad (14a)$$

$$\hat{y}_{t+h,c} = s_c \tilde{y}_{t+h,c} + \mu_c \quad (14b)$$

r parcourt le contexte ; $h = 1, \dots, H$. s_c est la racine de la variance du contexte augmentée de $\varepsilon > 0$. La forme affichée omet les paramètres affines appris.

Les statistiques proviennent uniquement du contexte. Une version affine ajoute une échelle et un décalage appris, inversés à la sortie. Cette normalisation facilite la comparaison de fenêtres de niveaux différents. Elle n'annonce toutefois pas une rupture future : elle replace les sorties à l'échelle du passé observé. Une expérience doit donc préciser si RevIN est présent dans tous les modèles comparés.

DLinear permet de vérifier l'utilité des blocs complexes. Le modèle calcule une tendance lissée et un résidu, applique à chacun une projection linéaire de L vers H , puis additionne les sorties. Il apprend ces projections en minimisant une erreur de prévision. Il n'y a ni tokens ni attention. Les résultats de Zeng et al. (2023) montrent l'intérêt de cette référence dans les configurations étudiées. Une comparaison utile conserve le même horizon et examine le réglage du contexte, car une différence de longueur d'entrée peut masquer l'effet propre de l'architecture.

DeepAR fournit un autre point de comparaison : un réseau récurrent partagé entre séries produit les paramètres d'une distribution conditionnelle. L'apprentissage maximise la vraisemblance des observations. À l'inférence, des valeurs sont échantillonnées successivement et réinjectées pour construire des trajectoires (Salinas et al., 2020). Cette procédure se distingue des sorties directes précédentes. Elle montre aussi que le partage de connaissances entre séries ne commence pas avec les Transformers.

3.8 Du modèle supervisé au modèle préentraîné

Les architectures décrites jusqu'ici répondent à des problèmes distincts. Modifier les tokens change les interactions accessibles ; sélectionner des requêtes change le calcul ; introduire une décomposition change les hypothèses sur le signal. Il est donc préférable d'étudier ces choix séparément, avec des ablations, avant d'attribuer un gain à la seule présence d'attention.

Le préentraînement ajoute une autre dimension : le modèle apprend sur un ensemble de séries avant son utilisation sur une tâche cible. Un objectif autorégressif apprend une continuation ; un objectif par masquage apprend à reconstruire des observations retirées. Les cibles sont construites à partir des séries elles-mêmes. Ces procédures restent des apprentissages avec une fonction de perte, même lorsqu'elles ne demandent aucune annotation manuelle.

En zero-shot, les paramètres restent fixes sur la tâche cible. En fine-tuning, une optimisation modifie tout ou partie de ces paramètres. Fournir davantage d'historique au modèle ne constitue donc pas un ajustement fin. Enfin, l'absence d'adaptation ne prouve pas que le modèle n'a jamais rencontré les données cibles pendant son préentraînement. Cette différence devient centrale lorsque l'on passe des architectures supervisées aux modèles de fondation.

Un modèle de fondation peut rester gelé alors qu'une régression auxiliaire, une correction résiduelle ou des poids d'ensemble sont ajustés sur la tâche cible. Il faut alors distinguer l'absence d'adaptation du réseau préentraîné de l'apprentissage réalisé par le système complet. La qualification zero-shot doit préciser le composant auquel elle s'applique.

LoRA conserve les poids préentraînés W_0 et apprend une correction de faible rang $\Delta W = BA$. Pour une matrice de taille $d \times k$, seules deux matrices totalisant $r(d + k)$ paramètres sont entraînées, avec r très inférieur aux dimensions initiales (Hu et al., 2022). Cette économie facilite l'adaptation ; celle-ci reste un apprentissage sur les données cibles, même lorsque W_0 demeure figée.

3.9 Mesurer la qualité des prévisions

Sur N observations, la MAE moyenne les erreurs absolues ; la RMSE prend la racine de la moyenne des erreurs quadratiques. Les deux conservent l'unité de la cible, mais la RMSE pénalise davantage les grandes erreurs. La perte absolue cible une médiane conditionnelle, la perte quadratique une moyenne : le score doit donc être lu avec la sortie évaluée (Hyndman et Athanasopoulos, 2021).

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|, RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2} \quad (15)$$

La MASE rapporte la MAE de test à une erreur naïve calculée sur les T observations d'entraînement. Les différences sont espacées de m pas, avec m égal à la période saisonnière, ou à 1 sans saisonnalité ; T doit dépasser m (Hyndman et Koehler, 2006 ; Hyndman et Athanasopoulos, 2021). Une MASE inférieure à 1 signifie un gain face à cette référence historique, sans garantir un gain face à la naïve sur le test.

$$MASE = \frac{\frac{1}{H} \sum_{h=1}^H |y_{T+h} - \hat{y}_{T+h}|}{\frac{1}{T-m} \sum_{t=m+1}^T |y_t - y_{t-m}|} \quad (16)$$

Un dénominateur nul ne donne pas de MASE finie exploitable, et 0/0 reste indéfini. Une valeur presque nulle rend aussi le score instable. Ces cas et leur traitement doivent être signalés ; ajouter une constante change la mesure.

Pour un quantile de niveau τ , la perte pinball pondère une sous-estimation par τ et une surestimation par $1 - \tau$. Son espérance est minimisée par un quantile de la cible. Les niveaux évalués et leur agrégation doivent être précisés (Gneiting et Raftery, 2007).

$$\rho_\tau(y - q) = (y - q)(\tau - \mathbf{1}_{\{y < q\}}), 0 < \tau < 1 \quad (17)$$

Le CRPS intègre l'écart quadratique entre la probabilité $F(z)$ et l'indicatrice de $y \leq z$. Plus il est faible, meilleur il est. Pour les distributions de premier moment fini, il est strictement propre : annoncer la distribution réelle minimise le score espéré. Une prévision déterministe ramène le CRPS à l'erreur absolue (Gneiting et Raftery, 2007).

$$CRPS(F, y) = \int_{-\infty}^{+\infty} \left[F(z) - \mathbf{1}_{\{y \leq z\}} \right]^2 dz \quad (18)$$

Lorsqu'un modèle fournit seulement une grille finie de quantiles, le score calculé à partir de cette grille constitue une approximation du CRPS ou une perte quantile agrégée, selon la convention retenue. Les niveaux, pondérations, normalisations et règles d'interpolation doivent être identiques ou explicitement documentés pour comparer les modèles (Gneiting et Raftery, 2007 ; Shchur et al., 2025).

Ces scores évaluent les marges, sans vérifier la dépendance entre dates. Des scénarios complets demandent également une évaluation de leur loi jointe.

4 Méthodologies des modèles de fondation temporels

4.1 Du préentraînement à une nouvelle prévision

Les modèles de fondation temporels cherchent à réutiliser un apprentissage réalisé sur de nombreuses séries. Une fenêtre d'historique devient l'entrée du modèle ; les observations suivantes fournissent la cible d'apprentissage. En répétant cette opération sur des domaines et des fréquences variés, le réseau apprend une fonction de prévision partageable. Lorsqu'il reçoit une nouvelle série, son contexte change, mais ses paramètres peuvent rester fixes : c'est le fonctionnement en zero-shot. Une adaptation par gradient constitue une étape différente, qui doit être annoncée séparément.

Pour comparer les méthodes, on retient quatre questions : comment les nombres sont-ils représentés, quelles informations peuvent interagir, quelle perte est minimisée et comment le futur est-il construit ? La sortie mérite une attention particulière. Une valeur centrale, neuf quantiles et plusieurs trajectoires ne décrivent pas exactement le même objet statistique. Le choix de l'architecture doit donc être lu avec celui de l'objectif et du décodage.

La littérature retenue s'arrête au 9 septembre 2026. Les méthodes fondatrices de TimesFM et Moirai sont publiées à ICML 2024 ; Time-MoE à ICLR 2025 ; Sundial et Moirai-MoE à ICML 2025 ; TiRex à NeurIPS 2025. Chronos-2, Moirai 2.0, Timer-S1 et Toto 2.0 sont étudiés dans les versions de leurs rapports techniques référencées en bibliographie. Pour TimesFM-3, la source utilisée est une annonce officielle datée du 31 août 2026. Ce dernier cas permet de discuter la méthode annoncée, avec un recul expérimental plus limité.

4.2 Chronos et TimesFM face à la représentation des valeurs

Chronos transforme la prévision en classification séquentielle. Chaque série est divisée par la moyenne des valeurs absolues de son contexte, puis les nombres normalisés sont associés à des classes d'un vocabulaire fini. Ces identifiants alimentent notamment une architecture T5 encodeur-décodeur, entraînée sur des séries temporelles depuis une initialisation aléatoire. Le modèle ne réutilise donc pas nécessairement des connaissances apprises sur du texte. Son objectif est l'entropie croisée des tokens futurs (Ansari et al., 2024).

$$\mathcal{L}_{CE}(\theta) = -\sum_{k=1}^K \log p_{\theta}(z_k \mid c, z_{1:k-1}) \quad (19)$$

K est ici le nombre de tokens futurs. La somme porte sur une séquence cible ; l'entraînement agrège cette perte sur les fenêtres du lot.

Dans cette écriture, c désigne le contexte tokenisé, K le nombre de tokens à prévoir, z_k le token futur observé et θ les paramètres. À l'entraînement, les tokens précédents corrects sont fournis au décodeur. À l'inférence, les nouveaux tokens sont échantillonnés puis réinjectés pour poursuivre la trajectoire. La conversion inverse vers des nombres et le rétablissement de l'échelle donnent une prévision. Plusieurs répétitions fournissent des scénarios et leurs quantiles. Cette méthode reste limitée par la finesse des classes, leur plage de valeurs et le coût des générations successives (Ansari et al., 2024).

Cette formulation pose une question de représentation : deux classes voisines correspondent à des nombres proches, mais l'objectif reste catégoriel. Le vocabulaire doit donc être étudié avec la remise à l'échelle.

TimesFM conserve au contraire des entrées continues. Des observations consécutives forment des patches, transformés en vecteurs par un petit réseau résiduel. Un codage de position leur

est ajouté, puis un Transformer à attention causale calcule leurs représentations. Une seconde projection transforme chaque représentation en bloc de valeurs futures. La taille du bloc de sortie peut dépasser celle du patch d'entrée : une unité interne peut donc apprendre à prévoir plusieurs unités futures. L'article initial optimise une erreur quadratique et se concentre sur la prévision ponctuelle (Das et al., 2024).

L'apprentissage masque aléatoirement le début du premier patch pour exposer le réseau à différentes longueurs de contexte, y compris celles qui ne sont pas des multiples de la taille des patches. À l'inférence, les blocs sont produits successivement si l'horizon demandé dépasse celui de la tête de sortie. Le regroupement réduit le nombre de tokens et d'appels, mais le décodage prolongé utilise encore des prévisions antérieures comme entrées. La robustesse à cette différence entre apprentissage et inférence reste donc à évaluer (Das et al., 2024).

TimesFM 2.5, diffusé en septembre 2025, compte 200 millions de paramètres et accepte jusqu'à 16 000 observations de contexte. La documentation décrit une tête optionnelle de 30 millions de paramètres pour prévoir des quantiles jusqu'à un horizon de 1 000 points. Les covariables sont prises en charge par XReg, un complément de régression au modèle temporel. Ces éléments décrivent une version logicielle et ses capacités documentées ; ils ne doivent pas être attribués sans distinction au protocole de l'article de 2024 (Google Research, 2026).

4.3 Moirai et Chronos-2 pour structurer les variables

Moirai 1 traite la prévision comme le remplissage d'une partie masquée. Les séries sont normalisées, découpées en patches et réunies dans une même séquence de tokens. Les patches futurs des cibles sont remplacés par une représentation apprise du masque ; les covariables effectivement connues restent disponibles. Un encodeur traite cet ensemble. L'attention ajoute un biais différent selon que deux tokens appartiennent à la même variable ou à deux variables distinctes, en complément du repérage temporel. Le modèle peut ainsi recevoir un nombre variable de composantes, sans associer chaque position de variable à une signification fixe (Woo et al., 2024).

La projection de sortie fournit les paramètres d'un mélange de distributions : Student, binomiale négative, lognormale et normale de faible variance. Le préentraînement minimise la log-vraisemblance négative des observations futures et fait varier contexte et horizon. Plusieurs tailles de patches sont proposées selon les fréquences. L'aplatissement des variables augmente toutefois le nombre total de tokens : avec C variables et N patches chacune, l'attention complète considère une séquence de longueur CN . Cette flexibilité a donc un coût lorsque C devient grand (Woo et al., 2024).

Moirai 2.0 modifie cette démarche. Les patches continus et leurs indicateurs de présence alimentent désormais un décodeur causal. La tête produit plusieurs patches futurs et neuf quantiles, entraînés avec la perte pinball. Le masquage aléatoire de patches cherche à rendre le modèle moins dépendant d'un contexte intégralement observé. Pour éviter une fuite par la normalisation pendant le préentraînement causal, les statistiques sont calculées sur les premiers 30 % de la séquence, les 70 % restants servant à la prédiction. Pour prolonger l'horizon, le rapport propose un décodage récursif à partir des quantiles (C. Liu et al., 2025).

Cette simplification s'accompagne d'une restriction : les variables sont traitées indépendamment et le support des covariables est retiré. Moirai 2.0 ne représente donc pas une extension de toutes les capacités de Moirai 1. Le changement de méthode privilégie la prévision univariée probabiliste et l'efficacité. Une grille de quantiles renseigne les marges, mais ne

définit pas à elle seule une distribution jointe de toutes les dates ; la procédure utilisée pour prolonger les sorties devient déterminante (C. Liu et al., 2025).

Chronos-2 rompt aussi avec la représentation de sa première génération. Les valeurs sont standardisées, transformées par la fonction sinus hyperbolique inverse (arsinh), puis regroupées en patches continus avec un indice temporel et un masque d'observation. Les futures cibles sont indiquées comme manquantes. L'architecture est un encodeur seul qui alterne attention temporelle au sein d'une série et attention entre les séries d'un même groupe, au même indice de patch. Un groupe peut réunir plusieurs cibles et leurs covariables. La tête fournit directement les quantiles des patches futurs, et la perte pinball porte uniquement sur les cibles observées de l'apprentissage (Ansari et al., 2025).

Les covariables catégorielles sont converties en valeurs numériques avant l'entrée dans le réseau : target encoding pour une cible univariée, encodage ordinal dans le cas multivarié décrit par l'article. Cette prise en charge repose sur un encodage numérique des catégories, sans traitement natif de covariables textuelles (Ansari et al., 2025).

Le partage d'information entre variables modifie le conditionnement des prévisions. Il ne suffit pas à définir la dépendance des erreurs futures : une tête de quantiles fournit encore des quantiles marginaux, par variable et par horizon. Cette distinction sera reprise dans la discussion des trajectoires probabilistes en section 5.5 (Ansari et al., 2025 ; Perez-Diaz et al., 2025).

Les tâches multivariées du préentraînement sont construites synthétiquement en imposant des dépendances entre séries. L'enjeu est d'apprendre à exploiter une relation visible dans le contexte sans réentraîner le réseau sur chaque tâche. Le masque précise quelles valeurs futures sont connues ; l'identifiant de groupe précise quelles séries peuvent échanger de l'information. La présence de ces mécanismes ne garantit cependant pas que les dépendances du corpus synthétique couvrent celles d'un nouveau domaine (Ansari et al., 2025).

L'annonce de TimesFM-3 décrit une autre combinaison : patches de 32 points, décodeur, attention temporelle causale et attention entre variables alternées. Les tokens des covariables connues dans le futur incluent des patches à venir. Le modèle reçoit aussi des patches futurs masqués et fournit neuf quantiles par cible. Son préentraînement est présenté comme nativement multivarié, sur des données réelles et synthétiques, avec un décodage de l'horizon en une passe. La publication consultée décrit ces choix, sans documenter une recette d'optimisation aussi complète qu'un article méthodologique. On retient donc cette évolution comme une piste récente, sans reprendre ses classements comme preuve indépendante de supériorité (Jain et Sen, 2026).

4.4 Sundial et la génération continue du futur

Une prévision probabiliste peut aussi être obtenue en apprenant une transformation entre du bruit et des observations. Le flow matching entraîne un réseau à prédire un champ de vitesse : à partir d'un point intermédiaire, il indique dans quelle direction le déplacer. Pour comprendre le principe, on considère un chemin linéaire entre un bruit gaussien ε et un bloc réel y . Le paramètre τ appartient à l'intervalle $[0, 1]$; il mesure l'avancement de cette transformation et ne représente pas une date de la série (Lipman et al., 2023).

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{\tau, \varepsilon, y, h} [\|v_{\theta}(x_{\tau}, \tau, h) - (y - \varepsilon)\|_2^2] \quad (20)$$

τ est uniforme sur $[0, 1]$ et ε est un bruit gaussien $N(0, I)$. h désigne ici la représentation du passé. La norme euclidienne est mise au carré.

Le point intermédiaire x_τ vaut $(1 - \tau)\varepsilon + \tau y$. La vitesse cible le long de ce chemin est $y - \varepsilon$. Le réseau v_θ reçoit x_τ , τ et, dans le cas conditionnel, une représentation h du passé. La perte compare sa vitesse à la vitesse cible. Lors de la génération, y est inconnu : on part d'un nouveau bruit et l'on intègre numériquement le champ appris. Changer le bruit initial produit un autre échantillon, pour un même contexte. Le préentraînement n'exige pas de simuler toute l'intégration à chaque mise à jour (Lipman et al., 2023).

Sundial applique ce principe à la prévision temporelle avec la perte TimeFlow. Il normalise la série, encode ses patches continus et calcule h avec un Transformer causal. Un petit réseau de flow matching, entraîné conjointement, transforme ensuite un bruit en bloc futur conditionné par h . Le même historique encodé est réutilisé pour générer plusieurs échantillons. L'entraînement porte sur TimeBench, un corpus comprenant des observations réelles et synthétiques. Les trajectoires permettent d'estimer médiane, quantiles et autres statistiques (Y. Liu et al., 2025).

Le décodage comporte ainsi deux progressions : les étapes numériques qui produisent un bloc, puis la prolongation temporelle si plusieurs blocs sont nécessaires. L'intérêt est de modéliser des sorties continues sans quantification ni mélange fixé de lois simples. En contrepartie, il faut choisir le nombre d'étapes d'intégration et d'échantillons. Ces choix influencent le coût et l'approximation obtenue. Sundial suit par ailleurs un préentraînement univarié : une génération probabiliste souple ne signifie pas automatiquement que les dépendances entre variables sont apprises (Y. Liu et al., 2025).

4.5 Les mélanges d'experts et la spécialisation des calculs

Dans un mélange d'experts, ou MoE, le réseau ne mobilise qu'une partie de ses sous-réseaux pour chaque token. Le routeur calcule des scores, sélectionne quelques experts et combine leurs sorties. La capacité stockée peut ainsi être élevée alors que le calcul par token reste limité. Cette séparation concerne surtout les transformations internes ; elle ne dispense pas de calculer les interactions temporelles de l'attention.

Time-MoE représente chaque point par un vecteur continu et utilise un décodeur causal. Dans ses couches, un ensemble d'experts remplace le réseau feed-forward classique. Un expert partagé est complété par des experts sélectionnés selon leurs scores. Plusieurs têtes prédisent des horizons différents ; l'apprentissage agrège leurs pertes de Huber, quadratiques près de zéro et linéaires pour les grandes erreurs, avec une pénalisation destinée à équilibrer l'utilisation des experts. Le corpus Time-300B contient plus de 300 milliards de points (Shi et al., 2025).

À l'inférence, l'organisation des différentes têtes permet de construire l'horizon demandé. Il s'agit principalement de prévisions ponctuelles, et les variables sont traitées indépendamment. Le routage peut se concentrer excessivement sur certains experts, d'où la perte auxiliaire. Par ailleurs, un token par observation conserve une séquence plus longue qu'une représentation par patches. Le nombre de paramètres activés ne résume donc ni la mémoire nécessaire ni le coût complet de la prévision (Shi et al., 2025).

Moirai-MoE utilise au contraire des patches et une projection d'entrée commune aux fréquences. Son décodeur causal apprend à prédire la distribution du patch suivant. Des couches MoE remplacent les feed-forward et orientent les tokens vers des experts spécialisés. Le travail étudie notamment un routage guidé par des centres de groupes obtenus à partir de représentations préentraînées. La tête produit les paramètres d'un mélange de distributions,

entraîné par log-vraisemblance négative. La spécialisation se fait ainsi au niveau des motifs représentés, au lieu de dépendre uniquement d'une catégorie de fréquence (X. Liu et al., 2025).

L'interprétation demande néanmoins de la prudence. Un expert sollicité sur certains patches n'est pas forcément un détecteur d'une composante bien identifiée, comme la tendance. Pour étayer cette lecture, il faudrait vérifier que les affectations se maintiennent après un changement d'échelle, un rééchantillonnage ou l'ajout de bruit. Ce type d'analyse aide à distinguer une spécialisation utile d'un simple découpage accidentel du corpus.

4.6 La construction des horizons longs

Timer-S1 déplace une partie de la prévision successive dans la profondeur du modèle. Après normalisation et projection de patches univariés, des blocs TimeMoE calculent les représentations. Des blocs TimeSTP les reprennent ensuite en consultant aussi les entrées initiales ; chaque étape ajoute un décalage de prévision. Une tête partagée fournit des quantiles. L'horizon long bénéficie ainsi de transformations sérielles supplémentaires à l'intérieur d'un appel, sans réintroduire systématiquement chaque valeur prévue dans le contexte brut (Y. Liu et al., 2026).

Le préentraînement est suivi d'une poursuite de l'apprentissage avec une pondération privilégiant les horizons proches, puis d'une extension du contexte. Le rapport présente 8,3 milliards de paramètres au total, dont 0,75 milliard activé par token. Cette capacité reste orientée vers des séries traitées indépendamment. La méthode permet surtout d'étudier où placer le calcul : dans plusieurs appels, plusieurs sorties parallèles ou plusieurs transformations successives. Une seule passe ne signifie donc pas un calcul sans dépendances internes (Y. Liu et al., 2026).

TiRex apporte un contrepoint avec une architecture xLSTM. Des patches, accompagnés d'indicateurs de valeurs manquantes, alimentent un réseau récurrent ; une tête apprend neuf quantiles avec la perte pinball. Le préentraînement masque des blocs consécutifs. À l'inférence, les patches futurs restent indiqués comme manquants et l'état interne propage l'information nécessaire à la prévision, au lieu de recevoir systématiquement la médiane prédite comme une observation. La mémoire récurrente remplit donc une fonction différente de la mémoire accessible par attention (Auer et al., 2025).

Toto 2.0 reprend le masquage contigu dans un Transformer par patches dont les attentions temporelle et intervariables alternent. La normalisation robuste et les projections résiduelles préparent les entrées ; une tête produit neuf quantiles avec une perte pinball, remplaçant le mélange de Student de la génération précédente. Pendant l'apprentissage, des portions consécutives sont masquées ; à l'inférence, elles correspondent à l'horizon à prévoir. Les sorties peuvent être calculées en une passe ou par blocs successifs pour les horizons longs (Khwaja et al., 2026).

La famille est entraînée sur des séries d'observabilité et des données synthétiques, avec une procédure de transfert d'hyperparamètres entre tailles. Les grands modèles sont entraînés plus longtemps que les deux plus petits : leurs résultats ne constituent donc pas une comparaison à calcul constant. Le choix du masquage, de l'optimisation et du décodage participe aux effets observés ; attribuer toute amélioration à la seule taille du Transformer serait insuffisant (Khwaja et al., 2026).

Ces méthodes conduisent à une même exigence pour l'évaluation : définir ce qui est tenu constant. Une comparaison d'architectures doit conserver autant que possible les données,

l'information accessible, la cible statistique et le budget. Une comparaison de systèmes complets peut accepter leurs différences, mais doit les documenter. Cette distinction sera centrale pour interpréter les limites et les perspectives présentées dans les chapitres suivants.

5 Limites des modèles et de leur évaluation

Les résultats présentés jusqu'ici montrent l'intérêt des modèles de fondation, mais leur interprétation demande quelques précautions. Un bon score peut dépendre des données déjà rencontrées, des informations fournies au modèle ou de la métrique retenue. Ce chapitre examine ces difficultés à partir des travaux disponibles au 9 septembre 2026.

5.1 Vérifier ce que le modèle a déjà rencontré

Le zero-shot désigne généralement une prévision sans mise à jour des paramètres sur la tâche cible. Il ne garantit pas que les données étaient absentes du préentraînement. Meyer et al. distinguent le recouvrement d'échantillons et le chevauchement temporel entre séries corrélées. Deux séries différentes peuvent, par exemple, décrire le même événement exceptionnel. Contrôler les noms des jeux de données ne suffit donc pas : il faut aussi examiner leurs périodes et leurs relations. Cette analyse identifie un risque d'estimation optimiste ; elle ne signifie pas que tous les modèles sont contaminés (Meyer et al., 2025).

GIFT-Eval sépare l'exposition au segment d'entraînement de l'exposition au test. Dans sa documentation, l'étiquette zero-shot exclut les deux, tandis qu'un indicateur distinct signale la fuite vers le test. Ces catégories précisent la lecture du classement. Le dépôt fournit également un corpus de préentraînement conçu pour ne pas recouvrir son test. Il reste néanmoins nécessaire de distinguer la provenance déclarée des données d'une vérification indépendante (Salesforce AI Research, 2026).

TSFMAudit propose justement une méthode complémentaire. Les auteurs réalisent une courte adaptation exploratoire et observent conjointement la baisse de la perte et la modification du modèle. Une amélioration rapide avec peu de changements peut signaler une exposition antérieure. L'étude porte sur six modèles et 187 jeux de données. Ce signal reste à interpréter avec prudence : une tâche facile ou proche du domaine appris peut également être assimilée rapidement. Il permet d'orienter une vérification, sans établir à lui seul l'origine des données (Li et al., 2026).

5.2 Évaluer plusieurs tâches et des observations nouvelles

TIME rassemble 50 jeux de données et 98 tâches. Qiao et al. cherchent à renouveler les sources, contrôler leur qualité et analyser les résultats selon les propriétés temporelles. Cette démarche aide à repérer une difficulté sur les signaux irréguliers, par exemple, que masquerait une moyenne par domaine. Cependant, un jeu nouveau aujourd'hui peut intégrer le préentraînement d'un modèle ultérieur (Qiao et al., 2026).

Impermanent s'appuie sur des flux actualisés d'activité GitHub. Les prévisions sont évaluées au fil des nouvelles observations. Il devient ainsi possible de suivre la stabilité des performances et les changements de distribution. Le domaine reste particulier : une bonne généralisation sur l'activité de projets logiciels ne suffit pas à conclure pour toutes les séries temporelles (Garza et al., 2026).

On retient de ces travaux qu'il faut distinguer le transfert entre domaines sur des archives et la prévision d'observations réellement futures. Une dégradation sur un nouveau flux peut venir d'un changement du phénomène, sans révéler une contamination antérieure. Des références simples, évaluées sur les mêmes périodes, permettent de mieux interpréter cette dégradation.

5.3 Comparer les informations disponibles et les coûts

fev-bench comprend 100 tâches issues de sept domaines, dont 46 avec covariables. Ses scores de gain renseignent l'ampleur des écarts entre modèles. Ses intervalles reposent sur un bootstrap apparié des tâches et renseignent la stabilité des comparaisons lorsque la composition du benchmark varie. Ils ne quantifient pas, à eux seuls, toute l'incertitude liée aux observations futures ou aux répétitions d'entraînement. La comparaison suppose aussi des entrées comparables. Si un modèle reçoit une covariable future connue à l'origine et l'autre uniquement la cible, l'écart mesure l'ensemble du système, sans isoler l'effet de l'architecture (Shchur et al., 2025).

Guibert et al. comparent dix modèles sur deux ensembles de données en mesurant précision, durée et énergie, à contexte et horizon fixés. Les compromis observés encouragent à présenter le coût en regard de la qualité. Une mesure reste toutefois liée au matériel, au traitement par lots et aux opérations comptabilisées. Le coût d'une inférence ne couvre pas celui du préentraînement, de l'adaptation ou des essais d'hyperparamètres (Guibert et al., 2026).

5.4 Distinguer adaptation, oubli et robustesse

Lagil et al. comparent Chronos-2 et TTM-R3-PT gelés à un ajustement complet et à des adaptations LoRA, avec au maximum 4 000 étapes. Ils rapportent des améliorations après adaptation, tout en précisant que certains segments d'entraînement ont pu être rencontrés pendant le préentraînement. Comparer chaque modèle à sa propre version adaptée limite ce déséquilibre, mais n'établit pas un transfert vers un domaine entièrement inédit. Les conclusions restent liées aux deux modèles étudiés (Lagil et al., 2026).

L'apprentissage continu pose une autre difficulté. Karaouli et al. montrent qu'une amélioration sur une nouvelle tâche peut s'accompagner d'un oubli des précédentes. Pour évaluer des adaptations successives, il faut donc conserver des tâches anciennes et mesurer leur évolution. Une amélioration locale ne suffit pas à décrire la qualité du modèle après adaptation (Karaouli et al., 2025).

Enfin, Zhang et al. étudient six modèles soumis à des perturbations adversariales. Leur sensibilité varie notamment selon la position des observations modifiées et la longueur du contexte ; un ajustement adversarial peut la réduire dans le cadre étudié. Ces attaques intentionnelles ne reproduisent pas directement les valeurs manquantes ordinaires ou une dérive naturelle. Elles testent une fragilité précise, sans en donner la fréquence dans des conditions courantes (Zhang et al., 2025).

5.5 Tenir compte de la forme des prévisions

Perez-Diaz et al. rappellent que des distributions marginales ne déterminent pas la dépendance entre horizons. Deux variables de Bernoulli de probabilité un demi illustrent cette différence : elles valent simultanément un avec probabilité un quart si elles sont indépendantes, contre un demi si elles sont identiques. Les marges sont pourtant les mêmes. Une bande de quantiles ne permet donc pas, à elle seule, de construire des scénarios cohérents pour un maximum, une somme ou un dépassement persistant (Perez-Diaz et al., 2025).

Wan et al. examinent des prévisions ponctuelles presque plates, qui préservent mal certaines relations entre séries. Leur analyse relie ce comportement à la prévisibilité de la cible et à l'objectif d'apprentissage. Lorsque le signal prévisible est faible, une sortie lisse peut rester compatible avec la minimisation de l'erreur quadratique. Son apparence ne suffit donc pas à

juger le modèle : il faut vérifier les propriétés requises par la tâche, comme le classement entre séries (Wan et al., 2026).

6 Pistes de recherche et expériences proposées

6.1 Cadre commun

Les quatre hypothèses suivantes restent à tester. Avant l'accès au test, on fixerait les versions et empreintes des modèles, les partitions, les pertes principales, les budgets et les règles de décision. Les transformations seraient ajustées exclusivement sur les données admissibles. Les résultats seraient présentés par tâche puis agrégés à poids égal, avec analyses par domaine et régime.

Des intervalles à 95 % reposeraient sur un rééchantillonnage apparié de groupes de séries et de blocs temporels ; leur longueur serait choisie sur le développement. Des corrections pour les comparaisons confirmatoires multiples seraient prévues. Les marges proposées ci-dessous devraient être justifiées avant le test. Un intervalle recouvrant une limite serait non concluant : l'absence de réfutation ne vaut pas confirmation.

6.2 H1 L'avantage zero-shot persiste après le gel du modèle

On comparerait un modèle préentraîné gelé à une référence statistique sélectionnée puis figée sur le développement. Les observations seraient collectées après le gel, dans au moins trois domaines et à plusieurs fréquences. Chaque prévision serait horodatée avant réception de sa cible. Les modèles partageraient séries, dates, horizons et informations accessibles.

Pour chaque tâche, le gain serait $g = 1 - L_{\text{modèle}}/L_{\text{référence}}$, avec traitement séparé des pertes de référence nulles. Le critère principal serait le gain prospectif moyen. H1 serait soutenue si sa borne inférieure était positive, réfutée dans ce périmètre si sa borne supérieure était négative ou nulle, et non concluante sinon. La collecte suivrait toute la durée prévue. La comparaison avec des archives renseignerait la stabilité, sans attribuer automatiquement une baisse à la contamination.

6.3 H2 Les covariables apportent un gain malgré une qualité imparfaite

Un même modèle serait évalué avec la cible seule, des covariables correctes, retardées ou permutées par blocs. Une variable aléatoire servirait de contrôle négatif. Le conditionnement natif serait distingué d'une correction résiduelle apprise avant le test. Toute covariable future devrait être effectivement connue à l'origine ; sa présence dans un historique complet ne suffirait pas.

Les critères proposés seraient un gain d'au moins 2 % avec les bonnes variables et une dégradation d'au plus 5 % sous perturbation, relativement à la cible seule. H2 serait soutenue si les intervalles respectaient simultanément ces deux exigences, et réfutée si la borne supérieure du gain restait sous 2 % ou la borne inférieure de la dégradation dépassait 5 %. Un gain avec les contrôles négatifs imposerait d'examiner redondance, régularisation et fuite d'information.

6.4 H3 Une adaptation limitée réduit le coût sans perte excessive

À modèle initial et données identiques, on comparerait gel des paramètres, LoRA, correction résiduelle et ajustement complet. Une architecture compacte spécialisée compléterait les références. Les régimes seraient présentés successivement, avec retours à des régimes anciens. Après chaque adaptation, l'évaluation porterait sur la nouvelle tâche et sur un ensemble fixe de tâches antérieures.

On mesurerait temps total, mémoire maximale et, si possible, énergie, en incluant les recherches d'hyperparamètres. Les comparaisons à données identiques et à calcul identique seraient séparées. La non-infériorité proposée autoriserait au plus 2 % de perte supplémentaire face à l'ajustement complet ; l'oubli serait limité à 5 % relativement à l'état précédent. Les pertes nulles seraient traitées séparément. H3 exigerait que les bornes supérieures restent sous les deux marges et qu'un avantage de coût soit établi. Elle serait réfutée si une borne inférieure dépassait sa marge ou si le coût était clairement supérieur ou égal. Le nombre de paramètres entraînaux ne remplacerait pas la mesure.

6.5 H4 La dépendance améliore la prévision des événements multi-horizons

On comparerait indépendance, copule estimée sur les résidus historiques et dépendance d'un générateur natif. Pour isoler le couplage, les scénarios seraient transformés en rangs puis reconstruits sur une grille commune de quantiles par horizon. Les sorties natives non recalées seraient évaluées séparément. Une simulation à marges fixées vérifierait le protocole avant les données publiques.

Trois événements seraient définis avant le test : dépassement au moins une fois, dépassement persistant et somme supérieure à un seuil. Le critère principal serait la diminution absolue moyenne du score de Brier face à l'indépendance, à poids égal entre événements. Un score multivarié et des diagnostics de calibration complèteraient l'analyse. H4 serait soutenue par un gain dont la borne inférieure est positive, réfutée si sa borne supérieure est négative ou nulle, ou si une dégradation après changement de régime est établie. Le nombre de scénarios serait identique ; temps et stockage seraient mesurés.

6.6 Portée du programme

H1 à H4 évalueraient les limites discutées dans la revue : généralisation temporelle, qualité des covariables, compromis de l'adaptation et dépendance des prévisions. Le programme serait progressif : reproduction des comparaisons, vérification des contrôles, puis tests prospectifs. Les conclusions resteraient limitées aux modèles, tâches et régimes étudiés. Des résultats négatifs préciseraient eux aussi les conditions dans lesquelles une méthode plus simple reste préférable.

7 Conclusion Cette revue montre que l'analyse des Transformers temporels doit relier les représentations aux informations accessibles et aux sorties attendues. Le choix des tokens organise les interactions ; le préentraînement détermine les régularités rencontrées ; le protocole précise ce que mesure réellement un score. Une bonne performance ne suffit donc pas à identifier le mécanisme du transfert.

Les limites étudiées conduisent à privilégier des comparaisons contrôlées : données réellement futures, covariables disponibles à l'origine, budgets explicités et adaptation évaluée avec ses effets sur les tâches anciennes. Pour les prévisions probabilistes, les marges et la dépendance doivent être distinguées lorsque la décision porte sur plusieurs horizons.

Les quatre hypothèses proposées constituent un programme à réaliser. Elles visent à déterminer si les gains persistent sur des observations futures, résistent à une dégradation des covariables et restent utiles après adaptation. Elles interrogent également la valeur des dépendances entre horizons pour des événements définis avant le test. Leurs critères distinguent un résultat favorable, une réfutation dans le périmètre étudié et une expérience non concluante.

La contribution de ce mémoire est une synthèse critique du fonctionnement des modèles de fondation temporels et une démarche expérimentale explicite pour en étudier les limites. Les résultats cités appartiennent aux travaux examinés ; les protocoles proposés restent à mettre en œuvre. Cette distinction permet de formuler des questions de recherche précises sans présenter les possibilités suggérées par la littérature comme des résultats acquis.

Bibliographie

Les références sont classées par nom du premier auteur. Les statuts de publication et les versions précisent la portée des sources. Les sources en ligne du corpus initial ont été consultées les 9 et 10 septembre 2026.

Abdul Fatir Ansari ; Lorenzo Stella ; Caner Turkmen ; Xiyuan Zhang ; Pedro Mercado ; Huibin Shen ; et al. (2024). Chronos: Learning the Language of Time Series. Transactions on Machine Learning Research. DOI : [10.48550/arXiv.2403.07815](https://doi.org/10.48550/arXiv.2403.07815).

Abdul Fatir Ansari ; Oleksandr Shchur ; Jaris Küken ; Andreas Auer ; Boran Han ; Pedro Mercado ; et al. (2025). Chronos-2: From Univariate to Universal Forecasting. Prépublication, arXiv:2510.15821. DOI : [10.48550/arXiv.2510.15821](https://doi.org/10.48550/arXiv.2510.15821).

Andreas Auer ; Patrick Podest ; Daniel Klotz ; Sebastian Böck ; Günter Klambauer ; Sepp Hochreiter (2025). TiRex: Zero-Shot Forecasting Across Long and Short Horizons with Enhanced In-Context Learning. Advances in Neural Information Processing Systems, 38 (NeurIPS 2025), Main Conference Track. DOI : [10.52202/085713-1929](https://doi.org/10.52202/085713-1929).

Jimmy Lei Ba ; Jamie Ryan Kiros ; Geoffrey E. Hinton (2016). Layer Normalization. Prépublication, arXiv:1607.06450. DOI : [10.48550/arXiv.1607.06450](https://doi.org/10.48550/arXiv.1607.06450).

Rishi Bommasani et al. (2021). On the Opportunities and Risks of Foundation Models. Rapport de recherche, Stanford Center for Research on Foundation Models, arXiv:2108.07258. arXiv : [2108.07258](https://arxiv.org/abs/2108.07258).

Abhimanyu Das ; Weihao Kong ; Rajat Sen ; Yichen Zhou (2024). A decoder-only foundation model for time-series forecasting. Proceedings of the 41st International Conference on Machine Learning, PMLR, 235, 10148–10167. [Consulter la source](#).

Azul Garza ; Renée Rosillo ; Rodrigo Mendoza-Smith ; David Salinas ; Andrew Robert Williams ; Arjun Ashok ; Mononito Goswami ; José Martín Juárez (2026). Impermanent: A Live Benchmark for Temporal Generalization in Time Series Forecasting. Prépublication arXiv:2603.08707, version 2, 10 mars 2026. DOI : [10.48550/arXiv.2603.08707](https://doi.org/10.48550/arXiv.2603.08707).

Tilmann Gneiting ; Adrian E. Raftery (2007). Strictly Proper Scoring Rules, Prediction, and Estimation. Journal of the American Statistical Association, 102(477), 359–378. DOI : [10.1198/016214506000001437](https://doi.org/10.1198/016214506000001437).

Google Research (2026). TimesFM — dépôt officiel. GitHub, documentation consultée le 9 septembre 2026. [Consulter la source](#).

Loïc Guibert ; Benjamin Pasquier ; Frédéric Montet ; Beat Wolf (2026). Benchmarking Time Series Foundation Models on their Accuracy and Energy Consumption. Proceedings of the Fourth Swiss AI Days, Proceedings of Machine Learning Research, volume 309, pages 46–55. [Consulter la source](#).

Edward J. Hu ; Yelong Shen ; Phillip Wallis ; Zeyuan Allen-Zhu ; Yanzhi Li ; Shean Wang ; Lu Wang ; Weizhu Chen (2022). LoRA: Low-Rank Adaptation of Large Language Models. International Conference on Learning Representations (ICLR 2022). [Consulter la source](#).

Rob J. Hyndman ; Anne B. Koehler (2006). Another look at measures of forecast accuracy. International Journal of Forecasting, 22(4), 679–688. DOI : [10.1016/j.ijforecast.2006.03.001](https://doi.org/10.1016/j.ijforecast.2006.03.001).

Rob J. Hyndman ; George Athanasopoulos (2021). Forecasting: Principles and Practice. 3e édition, OTexts, Melbourne, Australie ; §5.8 « Evaluating point forecast accuracy ». [Consulter la source](#).

Ayush Jain ; Rajat Sen (2026). TimesFM-3: A zero-shot foundation model for multivariate forecasting. Google Research Blog, 31 août 2026. [Consulter la source](#).

Nouha Karaoui ; Denis Coquenot ; Elisa Fromont ; Martial Mermillod ; Marina Reyboz (2025). Foundation vs. Specialized Models: Evaluating Catastrophic Forgetting in Continual Time Series Forecasting. Prépublication arXiv:2510.00809 ; version 3 révisée le 26 juin 2026. DOI : [10.48550/arXiv.2510.00809](#).

Emaad Khwaja ; Chris Lettieri ; Gerald Woo ; Eden Belouadah ; Marc Cenac ; Guillaume Jarry ; et al. (2026). Toto 2.0: Time Series Forecasting Enters the Scaling Era. Rapport technique, arXiv:2605.20119. DOI : [10.48550/arXiv.2605.20119](#).

Taesung Kim ; Jinhee Kim ; Yunwon Tae ; Cheonbok Park ; Jang-Ho Choi ; Jaegul Choo (2022). Reversible Instance Normalization for Accurate Time-Series Forecasting against Distribution Shift. International Conference on Learning Representations (ICLR). [Consulter la source](#).

Morad Laghil ; Bertrand Pracca ; Emilie Devijver ; Eric Gaussier (2026). Foundation Models and Fine-Tuning: Toward a New Generation of Models for Time Series Forecasting. Prépublication arXiv:2607.23146, version 1, 25 juillet 2026. DOI : [10.48550/arXiv.2607.23146](#).

Hongkai Li ; Shifeng Xie ; Lefei Shen ; Zhuo Li ; Mouxiang Chen ; Xiaobin Zhang ; et al. (2026). TSFMAudit: Data Contamination Auditing in Forecasting Time Series Foundation Models. Prépublication arXiv:2605.26161, version 1, 24 mai 2026. DOI : [10.48550/arXiv.2605.26161](#).

Bryan Lim ; Sercan Ö. Arık ; Nicolas Loeff ; Tomas Pfister (2021). Temporal Fusion Transformers for interpretable multi-horizon time series forecasting. International Journal of Forecasting, 37(4), 1748–1764. DOI : [10.1016/j.ijforecast.2021.03.012](#).

Yaron Lipman ; Ricky T. Q. Chen ; Heli Ben-Hamu ; Maximilian Nickel ; Matt Le (2023). Flow Matching for Generative Modeling. International Conference on Learning Representations (ICLR 2023). DOI : [10.48550/arXiv.2210.02747](#).

Chenghao Liu ; Taha Aksu ; Juncheng Liu ; Xu Liu ; Hanshu Yan ; Quang Pham ; et al. (2025). Moirai 2.0: When Less Is More for Time Series Forecasting. Prépublication, arXiv:2511.11698. DOI : [10.48550/arXiv.2511.11698](#).

Xu Liu ; Juncheng Liu ; Gerald Woo ; Taha Aksu ; Yuxuan Liang ; Roger Zimmermann ; et al. (2025). Moirai-MoE: Empowering Time Series Foundation Models with Sparse Mixture of Experts. Proceedings of the 42nd International Conference on Machine Learning, PMLR, 267, 38940–38962. [Consulter la source](#).

Yong Liu ; Tengge Hu ; Haoran Zhang ; Haixu Wu ; Shiyu Wang ; Lintao Ma ; Mingsheng Long (2024). iTransformer: Inverted Transformers Are Effective for Time Series Forecasting. International Conference on Learning Representations (ICLR). [Consulter la source](#).

Yong Liu ; Guo Qin ; Zhiyuan Shi ; Zhi Chen ; Caiyin Yang ; Xiangdong Huang ; Jianmin Wang ; Mingsheng Long (2025). Sundial: A Family of Highly Capable Time Series Foundation Models. Proceedings of the 42nd International Conference on Machine Learning, PMLR, 267, 39295–39317. [Consulter la source](#).

Yong Liu ; Xingjian Su ; Shiyu Wang ; Haoran Zhang ; Haixuan Liu ; Yuxuan Wang ; et al. (2026). Timer-S1: A Billion-Scale Time Series Foundation Model with Serial Scaling. Rapport technique, arXiv:2603.04791, version 3. DOI : [10.48550/arXiv.2603.04791](#).

Marcel Meyer ; Sascha Kaltenpoth ; Kevin Zalipski ; Oliver Müller (2025). Rethinking Evaluation in the Era of Time Series Foundation Models: (Un)known Information Leakage Challenges. Prépublication arXiv:2510.13654 ; version 3 révisée le 25 février 2026. DOI : [10.48550/arXiv.2510.13654](#).

Yuqi Nie ; Nam H. Nguyen ; Phanwadee Sinthong ; Jayant Kalagnanam (2023). A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. International Conference on Learning Representations (ICLR). [Consulter la source](#).

Alvaro Perez-Diaz ; James C. Loach ; Danielle E. Toutoungi ; Lee Middleton (2025). Foundation Model Forecasts: Form and Function. Prépublication arXiv:2510.19345, version 1, 22 octobre 2025. DOI : [10.48550/arXiv.2510.19345](#).

Zhongzheng Qiao ; Sheng Pan ; Anni Wang ; Viktoriya Zhukova ; Yong Liu ; Xudong Jiang ; et al. (2026). It's TIME: Towards the Next Generation of Time Series Forecasting Benchmarks. ICML 2026, version camera-ready déposée sur arXiv:2602.12147v4 le 3 juin 2026. DOI : [10.48550/arXiv.2602.12147](#).

Salesforce AI Research (2026). GIFT-Eval — dépôt officiel et règles du classement. Documentation GitHub consultée le 9 septembre 2026. [Consulter la source](#).

David Salinas ; Valentin Flunkert ; Jan Gasthaus ; Tim Januschowski (2020). DeepAR: Probabilistic forecasting with autoregressive recurrent networks. International Journal of Forecasting, 36(3), 1181–1191. DOI : [10.1016/j.ijforecast.2019.07.001](#).

Oleksandr Shchur ; Abdul Fatir Ansari ; Caner Turkmen ; Lorenzo Stella ; Nick Erickson ; Pablo Guerron ; Michael Bohlke-Schneider ; Yuyang Wang (2025). fev-bench: A Realistic Benchmark for Time Series Forecasting. Prépublication arXiv:2509.26468 ; version 4 révisée le 30 juin 2026. DOI : [10.48550/arXiv.2509.26468](#).

Xiaoming Shi ; Shiyu Wang ; Yuqi Nie ; Dianqi Li ; Zhou Ye ; Qingsong Wen ; Ming Jin (2025). Time-MoE: Billion-Scale Time Series Foundation Models with Mixture of Experts. International Conference on Learning Representations (ICLR 2025). [Consulter la source](#).

Ashish Vaswani ; Noam Shazeer ; Niki Parmar ; Jakob Uszkoreit ; Llion Jones ; Aidan N. Gomez ; Łukasz Kaiser ; Illia Polosukhin (2017). Attention Is All You Need. Advances in Neural Information Processing Systems, 30, 5998–6008. [Consulter la source](#).

Shu Wan ; Miles Ma ; Hank Zhu ; Guangqi Liu ; Stephen Wang ; Qingsong Wen ; Huan Liu (2026). Forecast Collapse in Time-Series Foundation Models. Prépublication arXiv:2608.14106, version 1, 14 août 2026. DOI : [10.48550/arXiv.2608.14106](#).

Gerald Woo ; Chenghao Liu ; Akshat Kumar ; Caiming Xiong ; Silvio Savarese ; Doyen Sahoo (2024). Unified Training of Universal Time Series Forecasting Transformers. Proceedings of the 41st International Conference on Machine Learning, PMLR, 235, 53140–53164. [Consulter la source](#).

Haixu Wu ; Jiehui Xu ; Jianmin Wang ; Mingsheng Long (2021). Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting. Advances in Neural Information Processing Systems, 34, 22419–22430. [Consulter la source](#).

Ailing Zeng ; Muxi Chen ; Lei Zhang ; Qiang Xu (2023). Are Transformers Effective for Time Series Forecasting? Proceedings of the AAAI Conference on Artificial Intelligence, 37(9), 11121–11128. DOI : [10.1609/aaai.v37i9.26317](#).

Jiawen Zhang ; Zhenwei Zhang ; Shun Zheng ; Xumeng Wen ; Jia Li ; Jiang Bian (2025). Are Time-Series Foundation Models Deployment-Ready? A Systematic Study of Adversarial Robustness Across Domains. Prépublication arXiv:2505.19397. DOI : [10.48550/arXiv.2505.19397](#).

Haoyi Zhou ; Shanghang Zhang ; Jieqi Peng ; Shuai Zhang ; Jianxin Li ; Hui Xiong ; Wancai Zhang (2021). Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting. Proceedings of the AAAI Conference on Artificial Intelligence, 35(12), 11106–11115. DOI : [10.1609/aaai.v35i12.17325](#).